

Analyse Statique du Code (PSScriptAnalyzer) et CI/CD

Ce document aborde l'industrialisation du code PowerShell à travers l'analyse statique. Dans un écosystème DevOps / GitOps, la qualité du code n'est plus une option. Avant même d'être testé fonctionnellement ou déployé, un script doit respecter des standards stricts de sécurité, de lisibilité et de performance.

1. Le Concept du "Shift-Left" en DevOps

Dans l'administration système traditionnelle, les erreurs de syntaxe, les failles de sécurité ou les mauvaises pratiques (ex: mot de passe en dur) sont souvent découvertes **en production**, au moment de l'exécution du script.

La méthodologie DevOps impose le **Shift-Left** (Décalage vers la gauche).

Dans la frise chronologique du développement d'un script (Planification → Code → Test → Déploiement → Exploitation), le but est de **détecter les problèmes le plus à gauche possible**, c'est-à-dire directement sur le poste de l'administrateur ou dès le premier *commit* Git, bien avant le déploiement.

Pour accomplir cela, on utilise un outil d'analyse statique, communément appelé un **Linter**.

2. Présentation de PSScriptAnalyzer

PSScriptAnalyzer est le linter officiel de Microsoft pour PowerShell. C'est un moteur d'analyse statique : il n'exécute pas votre code, il lit le fichier texte (.ps1, .psm1) et vérifie si la syntaxe et les pratiques respectent des dizaines de règles d'ingénierie préétablies.

2.1 Installation

PSScriptAnalyzer est un module téléchargeable depuis la PowerShell Gallery :

```
Install-Module -Name PSScriptAnalyzer -Force -Scope CurrentUser
```

2.2 Les 3 Niveaux de Sévérité

Les règles analysées par le moteur sont classées en trois catégories :

1. **Error (Erreur)** : Une syntaxe illégale, une variable non définie ou un problème de sécurité critique (ex: Mot de passe en clair).
2. **Warning (Avertissement)** : Une mauvaise pratique qui risque d'entraîner des

comportements inattendus ou de casser la rétrocompatibilité (ex: Utilisation d'un alias comme ls au lieu de Get-ChildItem).

3. **Information** : Suggestions d'amélioration de la lisibilité ou de l'esthétique du code.

3. L'Audit de Code en Pratique (Invoke-ScriptAnalyzer)

La cmdlet principale du module est Invoke-ScriptAnalyzer.

3.1 Analyser un fichier spécifique

```
Invoke-ScriptAnalyzer -Path "C:\Projets\InfraTools\Public\Get-InfraDisque.ps1"
```

Sortie typique dans la console :

RuleName	Severity	ScriptName	Line	Message
PSAvoidUsingCmdletAliases	Warning	Get-Infra...	12	'where' is an alias of 'Where-Object'.
PSUseDeclaredVarsMoreThanAssignments	Warning	Get-Infra...	24	The variable 'Temp' is assigned but never used.
PSAvoidUsingWriteHost	Information	Get-Infra...	45	File contains 'Write-Host'. Use Write-Output or Write-Verbose instead.

3.2 Analyser un module complet

Vous pouvez lancer l'audit sur un dossier entier de manière récursive. Très utile pour valider un module avant de le publier :

```
Invoke-ScriptAnalyzer -Path "C:\Projets\InfraTools" -Recurse
```

4. Les Règles Fréquentes en Entreprise

Voici les règles les plus souvent déclenchées et pourquoi le linter exige leur correction.

PSAvoidUsingCmdletAliases (Bannir les Alias)

- **Pourquoi** : L'alias dir, ls, ou % est parfait pour taper vite en console. Dans un script sauvegardé, c'est interdit. Un autre administrateur (ou un Linuxien) pourrait ne pas connaître l'alias. Le code doit être descriptif (Get-ChildItem, ForEach-Object).
- **Correction** : Remplacer l'alias par le nom complet de la cmdlet.

PSAvoidUsingWriteHost (Bannir Write-Host)

- **Pourquoi** : Comme vu lors du J1S3, Write-Host détruit le flux d'objet et pollue l'écran sans

pouvoir être capturé par l'automatisation.

- **Correction** : Utiliser Write-Output (pour de la donnée) ou Write-Verbose / Write-Information (pour des logs).

PSUseApprovedVerbs (Verbes non conformes)

- **Pourquoi** : PowerShell maintient une nomenclature stricte. Si vous nommez votre fonction Ping-Server ou Check-Disk, l'analyseur vous bloquera.
- **Correction** : Utiliser Test-Connection (Verbe *Test*) ou Get-DiskHealth (Verbe *Get*).

PSAvoidUsingPlainTextForPassword (Sécurité critique)

- **Pourquoi** : La règle détecte des variables nommées \$Password ou \$Cred auxquelles on affecte une chaîne de texte brute comme "P@sswOrd123".
- **Correction** : Utiliser Get-Credential ou le module SecretManagement (Vu au Jour 4).

5. Personnalisation et Fichier de Paramètres (Settings)

Chaque entreprise a ses propres standards. Il est possible de désactiver certaines règles du Linter (par exemple, si votre entreprise tolère exceptionnellement l'usage de Write-Host pour de simples scripts d'affichage).

Plutôt que d'exclure les règles manuellement dans la ligne de commande, la bonne pratique DevOps consiste à stocker les règles de l'entreprise dans un fichier de configuration (au format .psd1).

Création du fichier PSScriptAnalyzerSettings.psd1 :

```
@{
    # Inclure par défaut toutes les règles standards
    IncludeDefaultRules = $true

    # Liste des règles à ignorer pour ce projet
    ExcludeRules = @(
        'PSAvoidUsingWriteHost'
    )
}
```

Exécution avec les standards de l'entreprise :

```
Invoke-ScriptAnalyzer -Path ".\MonScript.ps1" -Settings ".\PSScriptAnalyzerSettings.psd1"
```

6. Intégration Continue (Le Pont vers la CI/CD)

C'est ici que l'analyse statique prend toute sa puissance DevOps.

PSScriptAnalyzer n'est pas conçu pour être exécuté uniquement "à la main" par le développeur. Il est le **gardien du dépôt Git**.

Le Pipeline de validation CI/CD

Dans un outil comme **GitLab CI** ou **GitHub Actions**, on configure un pipeline automatisé qui s'exécute à chaque fois qu'un administrateur fait un git push de ses scripts vers le serveur.

Le pipeline CI lance alors une machine jetable (Runner) et exécute PSScriptAnalyzer :

```
# Etape de Linting dans le pipeline CI
$Violations = Invoke-ScriptAnalyzer -Path ".\Src\" -Recurse | Where-Object Severity -in "Error",
"Warning"

if ($Violations) {
    Write-Error "ÉCHEC DE LA CI : Des erreurs de code ont été détectées."
    $Violations | Format-Table
    exit 1 # Le code de sortie 1 fait échouer le Job GitLab/GitHub
} else {
    Write-Host "SUCCÈS : Code conforme aux standards."
    exit 0
}
```

Conséquence : Si un script contient un alias ou un mot de passe en dur, la pipeline échoue (Statut : *Failed*), et le script ne sera jamais poussé en production. L'équipe d'exploitation garantit ainsi un niveau de qualité de code absolu.